Групповое задание. Срок сдачи – 3 недели. Разделение на группы по 2-3 человека. Результат работы должен быть опубликован в репозитории на GitHub/GitFlic/Bitbucket с конкретными коммитами от каждого участника группы. В репозитории должен лежать README с описанием задания на английском языке, примером входных и выходных данных, скриншотами результатов. На паре нужно продемонстрировать запуск программы, а также показать ее слабые места.

Примерный стек: Java, Gson, Java Collections Framework, Stream API. Использование UI приветствуется, но не обязательно. Можно использовать GPT-модели, в этом случае нужно подготовить выгрузку промптов и ответов, которые дала модель в удобочитаемом виде.

У конкурирующих групп одинаковый код не принимается, должны быть глобальные различия, не в названиях файлов или классов, а в реализации.

Одно и то же задание можно взять максимум три раза.

1. **Words from word**
   Нужно реализовать алгоритм/модуль, который по словарю и списку главных слов генерирует структуру уровней в JSON-формате.
   Уровень состоит из главного слова, набора подслов, сложности и минимального количества подслов, необходимых для прохождения.

   Входные данные:
   На вход подаются: dictionary — массив русских слов, например:
   ["десант", "адрес", "армия", "место", "метро"]

   mainWords — массив главных слов, для которых нужно построить уровни:
   ["ДЕМОНСТРАЦИЯ", "ВЗАИМНОСТЬ"]

   Генератор уровней принимает отдельный объект config, который управляет фильтрацией подслов, порогом прохождения и расчётом сложности. Конфиг не попадает в выходной JSON — он влияет только на генерацию.
   Конфиг:

```
{
 "minSubWordLength": 3,
 "minWordsToPass": 8,
 "difficultyThresholds": {
  "easyMaxMainWordLength": 7,
  "mediumMaxMainWordLength": 10,
  "easyMaxSubWords": 15,
  "mediumMaxSubWords": 30
 }
}
```

Поле	Тип	Назначение
minSubWordLength	number	Минимальная длина подслова. Слова короче не попадают в subWords
minWordsToPass	number	Минимальное количество подслов, которое нужно найти игроку для прохождения уровня. Также используется как порог генерации: если

		валидных подслов меньше, уровень не создаётся или помечается пропущенным
difficultyThresholds	object	Пороги для вычисления difficulty
easyMaxMainWordLength	number	Максимальная длина главного слова для уровня EASY
mediumMaxMainWordLength	number	Максимальная длина главного слова для уровня MEDIUM
easyMaxSubWords	number	Максимальное количество подслов для уровня EASY
mediumMaxSubWords	number	Максимальное количество подслов для уровня MEDIUM

Выходные данные:
JSON строго следующей структуры:

```
{
  "levels": [
    {
      "id": 1,
      "mainWord": "ДЕМОНСТРАЦИЯ",
      "difficulty": "HARD",
      "minWordsToPass": 8,
      "subWords": [
        { "text": "ДЕСАНТ", "hint": "" },
        { "text": "АДРЕС", "hint": "" }
      ]
    }
  ]
}
```

Поля уровня:

Поле	Тип	Описание
id	number	Уникальный идентификатор уровня
mainWord	string	Главное слово в верхнем регистре
difficulty	string	EASY / MEDIUM / HARD
minWordsToPass	number	Сколько подслов нужно найти для прохождения
subWords	array	Список подслов
subWords[].text	string	Подслово в верхнем регистре
subWords[].hint	string	Подсказка; на первом этапе допускается пустая строка

Правила генерации подслов
Подслово включается в уровень, если:
1. Оно есть в словаре.
2. Его длина не меньше minSubWordLength.
3. Оно не совпадает с главным словом.

4. Его можно составить из букв главного слова с учётом кратности букв. Пример: если в главном слове одна буква Я, то подслово с двумя Я недопустимо.
6. Оно не повторяется в рамках одного уровня.

Тестовые случаи
Проверить минимум следующие сценарии:
1. Главное слово с повторяющимися буквами.
2. Подслово использует букву больше раз, чем есть в главном слове.
3. Главное слово само присутствует в словаре.
4. Словарь содержит дубликаты.
5. Слова короче minSubWordLength.
6. Разный регистр и буква Ё.
7. Словарь без подходящих подслов.
8. Ровно minWordsToPass подслов.
9. Большой словарь — проверить производительность.

2. **Fillword**

Нужно создать алгоритм\модуль генерации структуры филворда.

Входные данные:
На вход подаются: dictionary — массив русских слов, например:
["ловкость", "кнут", "бабочка", "ад", "гиря"] , размер сетки: 3x3, 4x4, 5x5, 6x6

На выходе алгоритм формирует компактно расположенные слова в сетке, например как на Рисунке 1 и список с координатами клеток, по которым читается слово.
«Компактно» означает, что все слова должны помещаться в сетку заданного размера, сетка по итогу не должна содержать пустот и случайных символов.



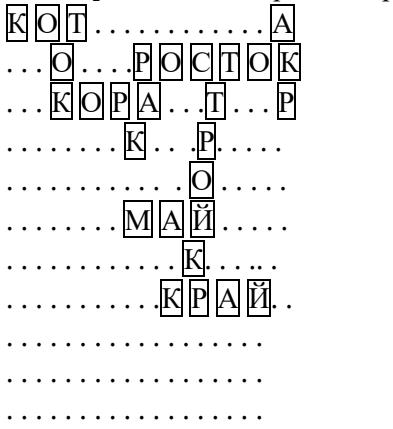
Рисунок 1 – Пример размещения слов в сетке 5x5
Слова: ловкость, кнут, бабочка, ад, гиря.

Правила размещения слов:
1. Каждое слово из словаря должно быть размещено ровно один раз.
2. Слово читается как последовательность соседних клеток:
    А) соседство только по стороне: вверх, вниз, влево, вправо;
    Б) диагональные переходы запрещены;
    В) слово может быть прямым или «ломаным», то есть может поворачивать.
3. Внутри одного слова одна и та же клетка не может использоваться дважды.
4. Разные слова не могут пересекаться в клетках.
5. Слова могут быть расположены в любом направлении: слева направо, справа налево, сверху вниз, снизу вверх и с поворотами.

3. **Crossword**

Нужно разработать алгоритм\модуль, который по массиву русских слов строит классический кроссворд в квадратной сетке заданного размера (5x5, 7x7, 9x9, 11x11) и возвращает координаты клеток, по которым читается каждое слово.
метод, который убирает пустые столбцы и строки, если они не заняты буквами слова.

Пример:
слова: ["кот", "ток", "рак", "кора", "акр", "росток", "май", "стройка", "край"]



Правила по выходным данным:

1. Слова не должны повторяться
2. Кроссворд должен состоять из 5+ слов
3. Слова должны пересекаться по общей букве
4. Отдельно стоящих слов не должно быть
5. Слова должны быть полностью на сетке без обрезки
6. Может быть такое, что слово из входящих данных выкидывается из кроссворда

4. Quiz

Нужно создать модуль генерации формы теста HTML + JavaScript

Предлагается подход на основе предметно-ориентированного языка (DSL). Синтаксис: ? — начало вопроса, r/c/t — тип ответа (одиночный выбор, множественный выбор, текстовый ввод), - — вариант, + — правильный ответ, e — конец блока вариантов, _ — текстовый ответ. Метаданные задаются директивами Title, H1, Style. Пример составления теста приведен в Листинге 1.

Title Текст, отображающийся на вкладке
H1 Название теста, которое будет видно на странице
Style Стиль элементов (по-умолчанию default)
? Пример задания с выбором одного варианта r
- Первый вариант
- Второй (правильный вариант) +
- Заключительный вариант e
? Пример задания с множественным выбором c
- 1
- 2 +
- 3
- 4 + e
? Вариант задания с текстовым вводом ответа t
_ Правильный ответ

Пример программы:

```
Title My title
H1 Super Test

? 2+2= r
- 4 +
- 5
- 6 e

? Second question c
- A
- B +
- C +e

? Text question t
_ My answer
```

Результат выполнения модуля:

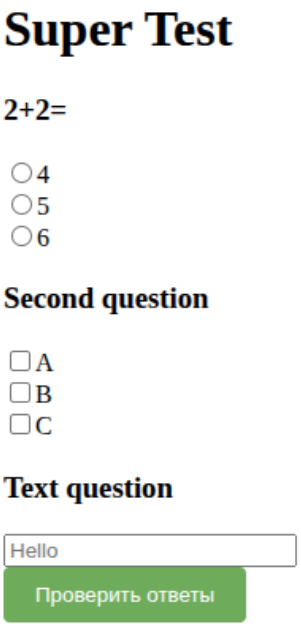


Рисунок 2 – пример генерации формы теста.

5. **Words**

Нужно создать алгоритм генерации классического сканворда, где слова будут расположены по горизонтали, вертикали и по диагонали, а остальные клетки заполнены случайными буквами. Пересечения слов по одинаковой букве необязательны. Минимальное количество слов на размещение: 10. Каждое слово встречается только один раз. Максимальная длина слова не должна превышать высоту, ширину или диагональ сетки. Размер сетки: 10x10, 11x10, 10x11, 12x10, 10x12.

Пример размещения:
Слова: май, жена, шляпа, пояс, кухня, ковер, бровь, директор, слово, март, учитель



Рисунок 3 - Пример размещения слов в сетке